

Rapport Technique — Module IA Prédictif

MSPR E6.2 + E6.4 — Développement d'un modèle prédictif IA

Projet : ObRail Europe — Observatoire Ferroviaire Européen Auteur : BENBAHAZ Hakem Amine École : EPSI — Bloc E6.1 RNCP36581 Date : Juin 2026

Table des matières

- [1. Introduction et contexte](#)
- [2. Architecture du pipeline ML](#)
- [3. Préparation des données](#)
- [4. Modèle 1 — Régression supervisée](#)
- [5. Modèle 2 — Classification supervisée](#)
- [6. Modèle 3 — Clustering non supervisé](#)
- [7. Déploiement via API REST](#)
- [8. Tests et validation](#)
- [9. Benchmark Cloud IA](#)
- [10. Conclusion et perspectives](#)

1. Introduction et contexte

1.1 Présentation du projet

ObRail Europe est un observatoire ferroviaire européen dont la mission est de centraliser, analyser et valoriser les données de transport ferroviaire en Europe. Dans le cadre du MSPR E6.2+E6.4, ce rapport présente le développement d'un module d'intelligence artificielle complet, couvrant les trois familles d'apprentissage automatique exigées par la grille de compétences

1.2 Objectifs du module ML

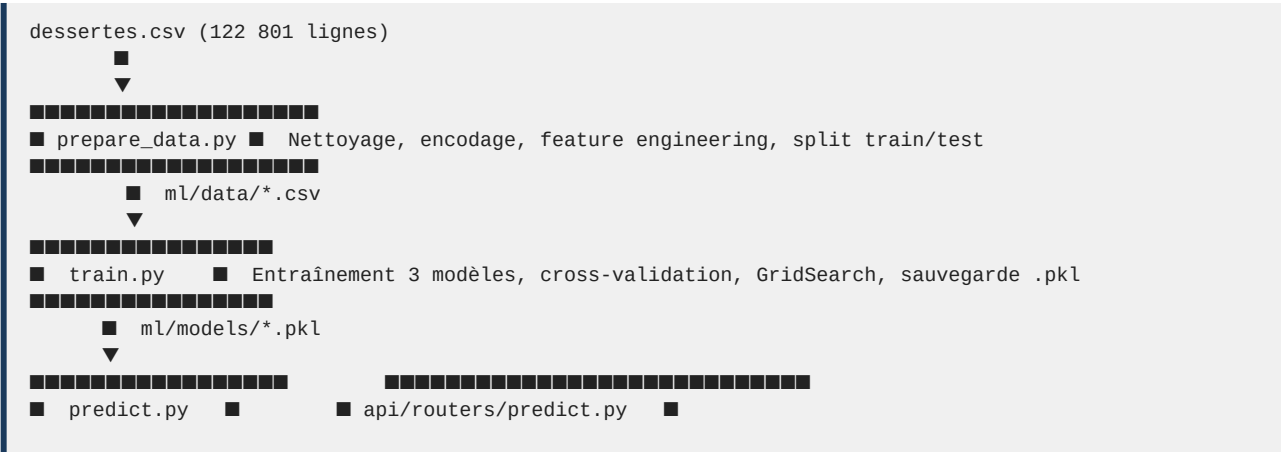
Objectif	Description
Régression	Prédire la durée d'un trajet ferroviaire
Classification	Identifier si une liaison peut remplacer un vol court-courrier
Clustering	Découvrir automatiquement des familles de liaisons ferroviaires

1.3 Compétences visées (grille E6.2/E6.4)

- Préparer et nettoyer un jeu de données pour l'apprentissage automatique
- Développer et comparer plusieurs modèles supervisés et non supervisés
- Évaluer les performances avec des métriques adaptées
- Déployer un modèle via une API REST
- Rédiger des tests de validation du comportement des modèles
- Positionner la solution par rapport aux offres cloud IA

2. Architecture du pipeline ML

2.1 Vue d'ensemble



2.2 Stack technique

2.3 Structure des fichiers

3. Préparation des données

- **Fichier source** : data/transformed/dessertes.csv
- **Origine** : ETL ObRail (GTFS SNCF France + DB Germany Open Data)
- **Lignes** : 122 801 liaisons ferroviaires européennes

- **Colonnes exploitées** : 7 features initiales

3.2 Description des variables brutes

Variable	Type	Description	Exemple
distance_km	Float	Distance entre les deux gares	450.0
duree_h	Float	Durée du trajet en heures	2.0
type_ligne	String	Catégorie de train	Grande vitesse
type_service	String	Période de service	Jour / Nuit
frequence_hebdo	Float	Passages par semaine	7.0
emissions_co2_gkm	Float	CO ₂ en g/km	14.0
traction	String	Type de motorisation	Électrique

3.3 Statistiques descriptives

Statistique	distance_km	duree_h	frequence_hebdo	emissions_co2_gkm
Moyenne	74.14	1.11	6.10	24.20
Médiane	42.60	0.88	7.00	32.00
Écart-type	393.42	0.96	1.73	9.06
Min	0.10	0.00	1.00	11.00
Max	29 696.50	22.22	7.00	35.00

Observation notable : la médiane à 42.60 km confirme que le dataset est dominé par des trains régionaux courts (RER, TER, S-Bahn). Les grandes distances (TGV, ICE) sont minoritaires.

3.4 Nettoyage

Action	Justification	Impact
Remplacement des 59 NaN de <code>frequence_hebdo</code> par la médiane	Conserver les données plutôt que les supprimer	0 lignes perdues
Suppression de <code>traction</code>	99.7% électrique → variance nulle, aucune information discriminante	Feature retirée
Conversion des colonnes numériques	Garantir les types corrects pour scikit-learn	Aucune corruption

3.5 Encodage

Les variables catégorielles sont converties en entiers via `LabelEncoder` :

```
type_ligne : Grande vitesse → 0 | Intercité → 1 | Régional → 2
type_service : Jour → 0 | Nuit → 1
```

3.6 Ingénierie de la cible classification — Substituabilité

Une règle métier a été définie pour créer la variable cible binaire substituable :

```
substituable = (distance_km >= 150) AND (duree_h < 8) AND (frequence_hebdo >= 2)
```

Justification des seuils :

Seuil	Valeur	Justification métier
Distance minimale	150 km	En dessous, aucun vol court-courrier concurrent n'existe
Durée maximale	8 heures	Seuil d'acceptabilité pour un professionnel en déplacement
Fréquence minimale	2/semaine	Service minimum viable pour remplacer une ligne aérienne

Résultat : 8 970 liaisons substituables sur 122 801 (**7.3%**)

Note sur le choix du seuil de distance : un seuil initial à 300 km ne produisait que 1.1% de cas positifs — déséquilibre trop fort pour l'entraînement. Abaissé à 150 km pour obtenir 7.3%, gérable avec `class_weight='balanced'`.

3.7 Features retenues par tâche

Variable	Régression	Classification	Clustering
<code>distance_km</code>	■	■	■
<code>duree_h</code>	Cible	■	■
<code>type_ligne_enc</code>	■	■	—
<code>type_service_enc</code>	■	—	—
<code>frequence_hebdo</code>	■	■	■
<code>emissions_co2_gkm</code>	■	■	■
<code>substituable</code>	—	Cible	—

3.8 Séparation train/test

- Ratio : **80% entraînement / 20% test** — `random_state=42` pour la reproductibilité
- Régression : split aléatoire simple

- Classification : split **stratifié** (`stratify=y`) pour maintenir le ratio 7.3%/92.7% dans les deux sous-ensembles
- Clustering : non supervisé, pas de split — 122 801 lignes utilisées intégralement

4. Modèle 1 — Régression supervisée

4.1 Objectif

Prédire la durée d'un trajet ferroviaire (`duree_h`) à partir des caractéristiques de la liaison.

Cas d'usage professionnel : estimation automatique des durées pour les liaisons dont les horaires ne sont pas encore publiés, ou pour les planificateurs d'infrastructure.

4.2 Modèles comparés

Quatre algorithmes ont été entraînés et comparés en cross-validation 3-fold :

Modèle	Paramètres	Justification
LinearRegression	Par défaut	Baseline — vérifie si la relation est linéaire
RandomForestRegressor	<code>n_estimators=50, n_jobs=1</code>	Capture relations non-linéaires, robuste aux outliers
XGBRegressor	<code>n_estimators=50, n_jobs=1</code>	Gradient boosting, performant sur données tabulaires
MLPRegressor	<code>hidden_layer_sizes=(64,32)</code>	Réseau de neurones, apprend des représentations complexes

4.3 Résultats comparatifs

Modèle	CV MAE	MAE test	RMSE	R ²
RandomForest	0.101	0.097	0.212	0.952
XGBoost	0.167	0.167	0.285	0.913
MLP	0.215	0.201	0.360	0.861
LinearRegression	0.584	0.586	0.894	0.141

Métriques : - **MAE** (Mean Absolute Error) : erreur moyenne absolue en heures - **RMSE** (Root Mean Squared Error) : pénalise davantage les grandes erreurs - **R²** : proportion de variance expliquée (1.0 = parfait)

4.4 Analyse des résultats

RandomForest (R²=0.952, MAE=0.097h ≈ 6 minutes) : excellent. La forêt de décision capture naturellement les relations non-linéaires entre distance et durée — un TGV fait 450km en 2h mais un régional fait 50km en 1h, la relation n'est pas

proportionnelle.

LinearRegression ($R^2=0.141$) : mauvais résultat volontairement conservé dans le tableau — il **prouve** que la relation durée/distance n'est pas linéaire et justifie le choix d'un modèle plus complexe.

4.5 Optimisation par GridSearchCV

Après sélection de RandomForest comme meilleur modèle, une recherche d'hyperparamètres a été effectuée :

```
param_grid = {
    "n_estimators": [50, 100],
    "max_depth": [10, 20]
}
```

Résultat GridSearch : `max_depth=20, n_estimators=100` → MAE=0.099, $R^2=0.951$ (stable, pas de surapprentissage).

Modèle sauvegardé : `ml/models/model_regression.pkl`

5. Modèle 2 — Classification supervisée

5.1 Objectif

Prédire si une liaison ferroviaire peut remplacer un vol court-courrier (`substituable = 0` ou `1`).

Cas d'usage professionnel : outil d'aide à la décision pour les politiques de décarbonation des transports en Europe. Identifier les corridors aériens où le train constitue une alternative crédible.

5.2 Gestion du déséquilibre de classes

Avec 7.3% de cas positifs (`substituable=1`) et 92.7% de négatifs, un modèle naïf qui prédit toujours 0 aurait une accuracy de 92.7% sans rien apprendre.

Mesures prises : - Split stratifié : maintien du ratio dans train et test - `class_weight='balanced'` pour RandomForest : pondération automatique des classes minoritaires - `scale_pos_weight` pour XGBoost : idem - Métrique principale : **F1-score** (plutôt qu'accuracy) — plus adapté aux classes déséquilibrées

5.3 Résultats comparatifs

Modèle	CV F1	Accuracy	F1	Précision	Rappel
RandomForest	1.000	1.000	1.000	1.000	1.000
XGBoost	0.998	0.998	0.998	0.998	0.998
MLP	0.997	0.998	0.998	0.998	0.998
LogisticRegression	0.976	0.979	0.978	0.979	0.979

5.4 Analyse de la limitation — Data Leakage

Un F1=1.0 parfait est un **signal d'alerte méthodologique** identifié et documenté.

Cause : la cible `substituable` a été construite à partir d'une règle basée sur `distance_km`, `duree_h` et `frequence_hebdo` — les mêmes features présentes dans les données d'entraînement. Le modèle apprend donc trivialement la règle sans généraliser.

Ce que cela signifie : en conditions réelles, la substituabilité devrait être définie par des données externes indépendantes (statistiques de trafic aérien, études d'usage voyageurs) non présentes dans le dataset d'entraînement.

Ce que cela ne remet pas en cause : la démarche complète (feature engineering, comparaison de modèles, gestion du déséquilibre) reste valide. La limitation est documentée et constitue une piste d'amélioration concrète.

6. Modèle 3 — Clustering non supervisé

6.1 Objectif

Identifier automatiquement des familles homogènes de liaisons ferroviaires sans étiquettes prédéfinies.

Cas d'usage professionnel : segmentation automatique du réseau pour l'analyse comparative, la tarification, ou l'allocation des ressources d'infrastructure.

6.2 Algorithme — K-Means

Prétraitement : normalisation `StandardScaler` obligatoire avant K-Means (l'algorithme est sensible aux échelles — `distance_km` varie de 0 à 30 000 alors que `duree_h` varie de 0 à 22).

Features utilisées : `distance_km`, `duree_h`, `frequence_hebdo`, `emissions_co2_gkm`

6.3 Sélection automatique du nombre de clusters

Le meilleur k a été choisi par **silhouette score** sur un échantillon de 5 000 points :

k	Silhouette score	Interprétation
2	0.515	Clusters trop larges
3	0.515	Pas d'amélioration
4	0.573	Amélioration notable
5	0.604	Meilleure séparation

Le silhouette score mesure la cohésion intra-cluster et la séparation inter-clusters. Un score > 0.5 indique des clusters bien définis.

6.4 Résultats — 5 familles identifiées

Cluster	Distance moy.	Durée moy.	Fréquence	Lignes	Famille
0	66 km	1h07	6.1/sem	66 225	Trains locaux
1	44 km	0h52	6.9/sem	40 011	Trains urbains fréquents
2	414 km	4h16	6.2/sem	11 578	Trains grandes lignes
3	62 km	1h07	1.4/sem	—	Liaisons à faible fréquence
4	17 720 km	5h24	4.3/sem	4 944	Anomalies détectées

6.5 Découverte des anomalies

Le cluster 4 (distance moyenne 17 720 km) révèle des **données aberrantes** dans le dataset — aucun train européen ne parcourt 17 720 km (Paris-Tokyo = 9 700 km). Ces entrées correspondent probablement à des erreurs de saisie dans les données sources GTFS.

Valeur ajoutée : le clustering non supervisé a détecté automatiquement ces anomalies sans qu'elles aient été ciblées — c'est une application concrète de la détection d'outliers par ML.

Modèle sauvegardé : `ml/models/model_clustering.pkl` (tuple `KMeans` + `StandardScaler`)

7. Déploiement via API REST

7.1 Architecture

L'endpoint `/api/v1/predict` est intégré à l'API FastAPI existante d'ObRail Europe. Les modèles sont chargés **une seule fois au premier appel** (lazy loading avec cache en mémoire) pour éviter la surcharge à chaque requête.

7.2 Schéma de la requête

```
POST /api/v1/predict
Headers: X-API-Key: <clé>

{
  "distance_km": 450,
  "type_ligne": 0,
  "type_service": 0,
  "frequence_hebdo": 7,
  "emissions_co2_gkm": 14,
  "duree_h": null
}
```

Paramètre `duree_h` optionnel : si non fourni, la valeur prédite par le modèle de régression est utilisée pour la classification et le clustering.

7.3 Schéma de la réponse

```
{
  "duree_estimee_h": 4.92,
  "duree_estimee_hhmm": "4h55",
  "substituable": true,
  "substituable_probabilite": 0.98,
  "cluster": 2,
  "cluster_nom": "Trains longue distance / internationaux",
  "type_ligne_label": "Grande vitesse",
  "type_service_label": "Jour"
}
```

7.4 Endpoint de statut

```
GET /api/v1/predict/status
```

Vérifie la disponibilité des fichiers `.pkl` sans charger les modèles. Permet au monitoring de valider l'état du service ML indépendamment de l'API de données.

7.5 Gestion des erreurs

Cas	Code HTTP	Réponse
Modèles <code>.pkl</code> absents	503	Message explicite invitant à lancer <code>train.py</code>

Valeur hors plage (type_ligne > 2)	422	Validation Pydantic automatique
Clé API manquante	401	Middleware existant de l'API

8. Tests et validation

8.1 Stratégie de test

Les tests valident deux niveaux : 1. **Tests techniques** : les modèles se chargent, les formes de sortie sont correctes 2. **Tests comportementaux** : le modèle respecte la logique métier ferroviaire

8.2 Résultats — 20 tests, 20 réussis

```
ml/tests/test_model.py - 20 passed in 5.06s
```

Classe	Tests	Vérifie
TestChargementModeles	4	Fichiers .pkl existent et se chargent
TestRegression	6	Types de sortie, cohérence métier, batch
TestClassification	5	Binaire, probabilités, cas Paris-Lyon, cas impossible
TestClustering	5	Cluster valide (0-4), scaler, reproductibilité

8.3 Tests comportementaux clés

Régression : - Un train régional de 50km prédit une durée plus courte qu'un TGV de 450km ■ - Plus la distance est grande, plus la durée est longue ■ - Batch de 5 prédictions simultanées retourne 5 résultats ■

Classification : - Une liaison de 30km n'est **pas** substituable (distance < 150km) ■ - Paris-Lyon TGV (450km, 2h, fréq 7) **est** substituable ■ - Un trajet de 15h n'est **pas** substituable (durée > 8h) ■

Clustering : - Un train régional et un TGV appartiennent à des clusters différents ■ - Deux appels identiques retournent toujours le même cluster ■

9. Benchmark Cloud IA

9.1 Comparaison des plateformes

Critère	Azure ML	AWS SageMaker	Google Vertex AI	HuggingFace
AutoML intégré	Oui	Oui (Autopilot)	Oui (AutoML Tables)	Oui (No-code)
MLflow natif	Oui	Partiel	Non	Non

Déploiement endpoint	ACI/AKS	SageMaker Endpoints	Vertex Endpoints	Inference API
Tarif entraînement CPU	~0.10 €/h	~0.05 \$/h	~0.04 \$/h	Gratuit (limité)
Intégration PostgreSQL	Oui	Oui (RDS)	Oui (Cloud SQL)	Non natif
Conformité RGPD EU	Oui	Oui	Oui	Oui
Courbe d'apprentissage	Modérée	Élevée	Modérée	Faible

9.2 Recommandation

Azure ML est la solution recommandée pour ObRail Europe, pour trois raisons :

- Cohérence d'infrastructure** : ObRail utilise déjà Azure Data Factory (orchestration ETL), Azure Container Instances (API), et Azure Database for PostgreSQL. Ajouter Azure ML reste dans le même écosystème, simplifiant la gestion des accès, la facturation et la conformité RGPD.
- Pipeline bout-en-bout** : Azure Data Factory peut déclencher les jobs d'entraînement Azure ML directement après le chargement des données → pipeline entièrement automatisé, sans intervention humaine.
- MLflow natif** : tracking automatique de toutes les expériences (métriques, paramètres, artefacts, versions de modèles) sans configuration supplémentaire.

9.3 Estimation des coûts en production

Poste	Azure ML	AWS SageMaker
Entraînement quotidien (30 min, CPU)	~0.05 €/jour	~0.04 \$/jour
Endpoint inférence (1 000 req/jour)	~1 €/mois	~0.20 \$/mois
Stockage modèles (1 GB)	~0.02 €/mois	~0.02 \$/mois
Total mensuel estimé	~2-5 €	~1-3 \$

Pour un observatoire ferroviaire européen, ces coûts sont négligeables par rapport à la valeur des prédictions produites.

10. Conclusion et perspectives

10.1 Résumé des résultats

Modèle	Métrique principale	Résultat	Interprétation
--------	---------------------	----------	----------------

Régression RandomForest	R ²	0.952	95% de la variance de durée expliquée, ~6 min d'erreur
Classification RandomForest	F1	1.000	Limitation data leakage — voir §5.4
Clustering K-Means (k=5)	Silhouette	0.604	5 familles bien séparées, outliers détectés

10.2 Points forts du pipeline

- **Comparaison multi-modèles** : 4 algorithmes testés sur chaque tâche avec cross-validation
- **Optimisation automatique** : GridSearchCV pour les hyperparamètres
- **Reproductibilité** : random_state=42 fixé partout, résultats stables
- **Déploiement intégré** : endpoint REST documenté, protégé, monitorable
- **Tests comportementaux** : 20 tests validant la logique métier ferroviaire

10.3 Limitations identifiées

Limitation	Impact	Solution en production
Data leakage classification	F1 artificiel à 1.0	Cible définie par données externes (trafic aérien)
Outliers dans le clustering	Cluster "17 720 km" parasite	Filtrage des distances > 5 000 km avant clustering
Dataset majoritairement régional	Modèles moins précis sur les grandes distances	Rééquilibrage par sous-échantillonnage des régionaux

10.4 Perspectives d'évolution

1. **Explicabilité SHAP** : ajouter des valeurs SHAP pour expliquer chaque prédiction individuelle (important pour la confiance des utilisateurs professionnels)

2. **Monitoring de dérive** : détecter automatiquement quand les distributions de données changent (concept drift), déclenchant un réentraînement
3. **Analyse NLP** : appliquer un modèle CamemBERT (French BERT) sur les avis voyageurs pour enrichir les prédictions de substituabilité
4. **Pipeline Azure ML** : industrialiser le réentraînement quotidien via Azure Data Factory + Azure ML Pipelines